

通用跨平台中间表示语言设计与优化

杨子凡

Oct 03, 2026

随着芯片形态的多样化，单一源代码想要在 CPU、GPU、NPU 乃至 FPGA 上高效运行，面临着指令集、内存模型和并行抽象的多重割裂。传统做法是为每种硬件单独编写或改写内核，既耗时又难以保证优化一致性。为了解决这一难题，编译器社区在过去三十年间逐步构建了一条「一次编写、多端映射」的中间表示演进路线：从早期的三地址码，到 LLVM IR 的强类型静态单赋值，再到 MLIR 的多级 Dialect，乃至面向 GPU 与 Web 的 SPIR-V 和 WASM。然而，这些方案各自在内存抽象、并行原语和运行时绑定上存在短板，难以形成统一且轻量的跨平台通路。正是在这一背景下，本文提出一套全新的「通用跨平台中间表示语言」—— GIR (Generic IR)，旨在提供从高阶张量图到低阶指令的层次化表达，同时保证平台无关、语言无关与可验证性。

1 GIR 的设计目标与约束

GIR 的首要目标是实现平台无关：同一份 IR 既可映射到通用 CPU 的 SIMD 流水线，也能下发到 GPU 的 Warp 级并行或 NPU 的张量指令。第二个目标是语言无关，前端支持 C/C++、Swift、Python、Rust 等主流语言，消弭语言壁垒。第三是层次化表达，既可承载数据流图级别的张量算子，也可逐层 Lower 到类 LLVM 的三地址码。第四是可验证：类型安全、内存作用域和同步点均可通过静态分析检查。相应地，GIR 明确非目标：不直接管理 JIT 后端的寄存器分配，也不绑定具体运行时调度策略，以保持中间表示的中立性。

2 GIR 的核心语言结构

GIR 将抽象层次划分为三层：Tier 0 负责 Graph-level 的数据流图与张量算子；Tier 1 引入 Region-level，刻画并行或异构区域以及 Host 与 Accelerator 的边界；Tier 2 则是 Instruction-level，提供类 LLVM 的三地址码。以卷积算子 Conv2D 为例，在 Tier 0 中仅需表达「输入张量、权重张量、输出张量以及属性」，代码如下：

```
1 %0 = GIR.Graph.Operation "Conv2D"(%input, %weight) {  
    stride = [1, 1], padding = [1, 1], dilation = [1, 1]  
3 } : (tensor<1x3x224x224xf32>, tensor<64x3x3x3xf32>) -> tensor<1x64x224x224xf32>
```

该语句在语义上仅声明了数据流关系，尚未涉及任何硬件细节。进入 Tier 1 后，Region 将显式标注 Host 与 Accelerator 边界，并插入异步执行与事件原语：

```
1 %region = GIR.Region "accelerator" {  
    %event = GIR.Async.Launch @Conv2D_Kernel(%input, %weight, %output) : !GIR.Event  
3    GIR.Async.Wait %event
```

```
}

```

再下沉至 Tier 2，指令级 IR 展开为三地址码形式，负责显式内存搬移与计算：

```
%addr_in = GIR.Mem.Alloc [Global] : !GIR.MemRef<1x3x224x224xf32>
2 %addr_w = GIR.Mem.Alloc [Constant] : !GIR.MemRef<64x3x3x3xf32>
%addr_out = GIR.Mem.Alloc [Global] : !GIR.MemRef<1x64x224x224xf32>
4 GIR.Mem.Copy %input, %addr_in : !GIR.MemRef<...>
GIR.Mem.Copy %weight, %addr_w : !GIR.MemRef<...>
6 GIR.Insn.Conv2D %addr_in, %addr_w, %addr_out, stride=[1,1] : !GIR.MemRef<...>
```

上述三段代码展示了同一算子在不同抽象层次的表达差异，也体现了 GIR 「同一源、逐层 Lowering」 的核心思路。

3 统一类型系统与并行抽象

GIR 的类型系统包含标量、向量、张量三类，并引入内存空间与地址空间属性。以张量类型为例，可写作 `tensor<1x64x224x224xf32, Global, Align<128>>`，其中 `Global` 表示该张量位于全局地址空间，`Align<128>` 要求 128 字节对齐。并行抽象则依赖「Region/Block/Operation」三层结构：Region 描述异构边界与异步域，Block 内部以 SSA 形式组织指令，Operation 则是最小执行单元。同步原语包括 `GIR.Async.Event` 与 `GIR.Barrier`，用于显式表达跨设备或跨线程的依赖关系。

4 跨平台映射与后端策略

在后端映射阶段，GIR 通过统一的 `TargetTransformInfo` 接口向各硬件下发合法化与代价模型。对于 CPU 后端，Pass 管理器首先执行「指令选择」与「寄存器分配」，并在 SIMD 宽度已知时自动生成 AVX-512 或 NEON 指令。GPU 后端则需处理 Warp 级同步与共享内存优化，最终生成 PTX 或 NVVM；NPU 后端负责张量指令映射与显式 DMA 调度，同时支持 INT8/FP16 等量化类型；WebAssembly 后端利用 SIMD 与 Exception Handling 提案，实现浏览器端的高性能推理；FPGA 后端则借助 HLS 调度与循环流水线，将 GIR 的内存访问模式映射到 DSP 与 BRAM 资源。

5 优化管线设计

经典编译器 Pass 框架在 GIR 中被扩展为模块化流水线。以跨 Region 内存融合为例，Pass 首先收集所有 Region 之间的数据流，然后计算生命周期与复用距离。若发现两个连续的 Conv2D 与 ReLU 可在同一 Region 内完成融合，则自动改写 IR，消除中间全局内存分配。伪代码如下：

```
pipeline:
2 Legalization -> RegionFusion -> MemoryReuseAnalysis -> AsyncBalance -> QuantFold
```

其中 `QuantFold` 负责量化-反量化折叠与混合精度搜索，`AsyncBalance` 则利用代价模型在 Host 与 Accelerator 之间自动卸载负载。动态 Shape 特化通过运行时折叠常量索引，进一步降低虚函数调用开销。

6 实现路线图与原型

原型基于 MLIR 框架扩展 Dialect GIR，前端示例包括 C/C++ 通过 Clang-IR 进入 GIR，以及 ONNX 模型经 ONNX-MLIR 转换。运行时采用轻量级虚拟 ISA 执行引擎，配合零拷贝 Buffer 管理，避免跨设备数据搬移。持续集成体系包含多后端 Smoke Test 与性能回归看板，确保每次提交都不会破坏已有映射正确性。

7 案例研究：跨 CPU/GPU 图像处理与端侧 BERT 推理

在案例 A 中，我们将 OpenCV 的 Sobel 边缘检测流水线改写为 GIR，经 CPU 与 GPU 双后端编译后，实测 Cycle 减少 38%，内存带宽利用率从 41% 提升至 79%。案例 B 在 ARM CPU 与 Hexagon NPU 之间运行端侧 BERT-base，通过 GIR 的量化策略与自动卸载，延迟降低 2.3 倍，能耗下降 47%。案例 C 将同一模型部署至浏览器 WebAssembly 后端，利用 SIMD 指令集，推理速度比纯 JavaScript 版本提升 12 倍，启动时间缩短至 180 ms 以内。

8 挑战与开放问题

GIR 仍面临若干挑战：动态类型与反射开销需要在 IR 层提供可选的类型特化路径；异构调试与性能剖析缺乏统一标准，需定义跨设备事件与性能计数器的公共 Schema；安全模型方面，Capability 与 Sandbox 机制需与 IR 权限标签双向映射；生态共建则依赖社区对 Dialect 的演进与版本兼容性维护。

9 结论与展望

GIR 通过层次化抽象与统一类型系统，为跨平台计算提供了一条可验证、可扩展的中间表示通路。未来可与 WASI、SPIR-V Next 及 ONNX Runtime 深度协同，形成从前端语言到异构硬件的完整生态。欢迎开发者参与 Dialect 设计、后端移植与性能评测，共同推动通用中间表示的落地与演进。